

Capstone Checkpoint 5.1: Multi-Agent Architecture and Coordination Plan

Author: Jacques Troussard

1. Problem and Multi-Agent Solution

The problem being solved is to automate the organization of incoming mail/important documents so that they are easily searchable, well understood (via extracted metadata), and securely stored.

Since we are trying to cover a few end goals, it would stand to reason that we could employ multiple agents to achieve them all, leading to our overall mission of getting paperwork—that messy kitchen counter full of mail—under control. This system fundamentally needs two different kinds of specialists. One needs more flexibility and creativity to tame the noisy inputs from any number of document styles, understand what the document is about, and identify what is important. The second will need to be more strict, following strongly typed rules that ensure the storage taxonomy is maintained and doesn't spiral out of control under the limitless deviations a model can take with each prompt and input.

Putting both responsibilities on a single agent leads to "cognitive bleed," where the LLM's generative flexibility causes folder synonym hallucinations (e.g., writing to "Personal/Utilities" in one run and "Home/utills" the next). Splitting these tasks between two distinct agents enforces a strict security boundary: one agent focuses purely on data extraction, while the other strictly polices directory rules and filesystem mutation.

2. System Scale and Agent Roles

I have limited the system to exactly 2 agents. Any further split (e.g., separate OCR, filing, or summary agents) introduces severe diminishing returns, massive token overhead, and unnecessary latency on local hardware.

Agent 1: The Extraction Agent (The Parser)

Reads the progressive OCR text stream. Maps unstructured details to a strict, intermediate Pydantic JSON template. Evaluates and assigns confidence scores to each extracted field (source, recipient, date) and passes the output downstream.

Agent 2: The Verification Agent (The Guardrail)

Acts as a strict policy validator. Receives the candidate JSON schema, validates confidence scores, cross-references entity names with existing database rules, enforces the anchor-key validation (account number matching), queries the local RAG database for past human corrections on low-confidence fields, and deterministically executes the local file move.

3. Coordination Strategy

As established previously, the linear ReAct pattern with minimal looping is the path forward for this project, which will limit the coordination strategy to that of a straightforward diagnostic tree. If X happens, move to step A; if Y happens, move to step B. More specifically, the extraction agent will take the input, rationalize as

much as it can about the document, and decompose its important parts into the correct forms. Once this extraction result is generated, the workflow moves everything to the verification agent. This is the more rules-based and strict agent. It will judge the output, fetch vector data, move streams into manual review if necessary, and finally determine its “real” life location within the file directory. If validation fails or confidence score thresholds are breached, the workflow terminates at a hard circuit breaker, dumping the document state into our persistent human manual review queue. There is no complex backtracking or state-looping between agents, keeping the process highly predictable and performant. However, a looping logic component could be installed to allow the extraction process to try to self-heal or improve its initial outputs with programmed config adjustments. I believe this would be appropriate for this system design—a small overhead cost for a chance to improve fidelity and lessen the processing work the verification agent will need to enact.

4. Communication Approach

Linear top-down with possible isolated iterations for generation fidelity improvements. Agents will not engage in unstructured natural language conversations or “brainstorming” loops, which are overly complex for an “identify and categorize” type application. Instead, communication is strictly structured. The Extraction Agent packages its results into a serialized, typed Pydantic JSON payload. This payload is passed as a static state dictionary to the Verification Agent. This agent will apply rules to the output and follow a limited prescription of tools and options for what path the workflow stream should take. This method minimizes token usage, keeps communication bounded, and allows us to easily write isolated unit tests for each handoff stage.

5. Trade-offs

Reliability vs. Latency: Separating the pipeline into two major steps introduces a slight computational step-barrier, but it drastically increases reliability. It guarantees that a probabilistic error during the extraction stage is caught and blocked by deterministic verification rules before any file is actually moved. Additionally, we are still considering the “manual review” feature of the application, but have since dropped the fine-tuning idea and instead leverage vector comparison where the manual review feedback will be used.

Complexity vs. Coordination Overhead: As much as I have tried to shoe-horn harness frameworks like CrewAI and LangChain into this capstone, I still fail to find an implementation that doesn’t over-tax itself in terms of compute or complexity. Very little benefit will come from this classification application, since we are only considering relatively static and uninteresting inputs: documents in pdf format, with a handful of encodings and for at least this version, one language support.

Scalability: This 2-agent architecture is highly scalable. The Extraction Agent is stateless and lightweight, meaning we can run multiple instances in parallel to parse batch-scanned folders concurrently. Meanwhile, a single centralized Verification Agent can process the serialized payloads sequentially against a locked SQLite transaction database.